# Day2 실습 서브노트
# 사내 규정 데이터로 LoRA SFT를 실제로 돌려보기

과목: sLLM 구현 및 Fine Tuning / 2일차
실습 모델: Qwen2.5-1.5B-Instruct + PEFT LoRA(r=16) + TRL SFTTrainer
돌려본 코드: scripts/train_lora.py (학습 완료), scripts/evaluate_model.py (평가 완료)
결과물: models/hr-qwen-lora/ (adapter 70.5 MB), outputs/training_log.json, outputs/evaluation_summary.json
실행 환경: Apple Silicon Mac, FP32 LoRA on MPS, PyTorch 2.13.0

---

Day1은 "왜 파인튜닝이 필요한가"를 확인하는 날이었고, 오늘은 그걸 실제로 돌리는 날이었다. 가이드2의 수업 권장 설정으로 `train_lora.py`를 그대로 실행했고, 32분 18초 뒤에 어댑터가 나왔다. 아래는 과제에 적힌 다섯 항목 순서대로, 내가 이해한 것과 실제 로그·산출물에서 확인한 숫자를 같이 적은 것이다. 학습이 끝난 뒤에는 로그와 어댑터 파일을 직접 열어봤는데, 거기서 설정 하나가 데이터의 4분의 1을 잘라내고 있었다는 걸 발견했다(부록 C). 그걸 근거로 평가 점수를 예측했고, 평가를 돌린 뒤 대조해 봤더니 **예측 두 개가 모두 지지되지 않았다.** 대신 예측하지 않았던 문제를 찾았다. 평가 코드의 생성 설정 하나가 모델 점수를 절반 아래로 떨어뜨리고 있었다. 그 과정을 4번에 순서대로 적었다.

---

## 1. 이해관계자 가치 (Pain Point)

Day1에서 확인한 문제는 "Base Qwen이 우리 회사 규정을 모른다"였다. 오늘 짚어야 하는 건 그 다음 질문이다. **모른다는 걸 알았으면, 왜 하필 사내에서 직접 학습시켜야 하는가.**

### 왜 상용 LLM을 붙이면 안 되는가

기술적으로는 상용 LLM을 붙이는 게 제일 빠르다. 성능도 1.5B보다 훨씬 좋다. 그런데 실습 데이터를 열어보면 왜 그게 선택지가 아닌지가 바로 보인다. 학습 데이터 97개 섹션에는 이런 것들이 들어 있다.

- 징계 절차, 보안 위반 조사 절차, 허위 신고 처리 기준
- 인사평가 절차, 성과급 산정 기준, 승진 기준
- BLUE 정보보안 등급 체계와 각 등급의 취급 제한
- 고객 계약상의 재택근무 제한 조건(AURORA-X)
- 퇴사자 자산 반납, 보안 의무, 학습비 환수 규정

이걸 외부 API에 프롬프트로 넣는다는 건, 회사의 징계 기준과 보안등급 체계와 고객 계약 조건을 매 질문마다 회사 밖으로 내보낸다는 뜻이다. 계약서에 "학습에 사용하지 않음"이 적혀 있어도 문제가 남는다. 학습 여부와 무관하게 **전송 자체**가 일어나고, 그게 실제로 지켜지는지를 우리가 검증할 방법이 없다. 보안·법무 입장에서 이건 성능 문제가 아니라 통제권 문제다.

그래서 방향이 반대로 잡힌다. 모델을 데이터가 있는 쪽으로 가져온다. 1.5B는 사내 장비에서 돌릴 수 있는 크기이고, 우리가 직접 학습시킬 수 있는 크기다. 교재에서 말한 **데이터 주권**이 추상적인 원칙이 아니라, 모델 크기를 1.5B로 고른 실질적인 이유였다.

> 수업 중 나온 질문: "Qwen2.5 말고 다른 모델 써도 되나요?" → 된다. 구조가 decoder-only(CLM)이고 PEFT가 붙는 모델이면 코드는 거의 그대로 돌아간다. Qwen2.5-1.5B-Instruct를 쓰는 이유는 한국어가 되고, 라이선스가 열려 있고, 실무에서 사내 배포용으로 실제로 많이 쓰는 크기이기 때문이다. 바꿔 말하면 "이 모델이어야만 하는" 이유는 없고, "이 조건을 만족하는 모델 중 하나"라는 뜻이다.

### 왜 검색(RAG)만으로 끝내지 못하는가

Day1에 이미 검색은 붙여봤다. 문서를 찾아서 프롬프트에 넣어주면 Base Qwen도 그 범위 안에서는 답했다. 그런데 거기서 세 가지가 남았다.

1. **조건 판단이 흔들린다.** "수료율 70%면 지원되나요"에서 검색은 정답 문서(80% 미만 제외)를 Top-1로 찾아왔는데, 모델은 "지원받을 수 있습니다"라고 답했다. 70 < 80이라는 산수를 놓쳤다.
2. **옆 문서를 섞어서 지어낸다.** 같은 답변에서 top_k=2로 딸려온 "연 120만원 한도" 문서를 끌어와 "다만 120만원보다 적게 지급될 것입니다"라는, 규정에 없는 문장을 만들어냈다.
3. **답변 형식과 거절 습관은 검색으로 안 잡힌다.** 근거를 먼저 대고, 절차를 순서대로 쓰고, 규정에 없으면 없다고 말하는 건 지식이 아니라 습관이다.

검색은 *모델 밖에 있는 지식*을 붙이는 방법이지, *모델의 답변 습관*을 바꾸는 방법이 아니다. 이 습관을 바꾸는 게 SFT이고, 그 SFT를 15억 개를 다 건드리지 않고 하는 방법이 LoRA다. 오늘 실습이 정확히 그 지점이었다.

### 누구의 불편이고, SFT가 무엇을 주는가

| 이해관계자 | 지금 겪는 일 | SFT를 적용하면 달라지는 것 |
|---|---|---|
| 현업 직원 | 규정을 물어보면 그럴듯한 일반론이 돌아온다. 그게 우리 회사 규정인지 아닌지 본인이 판단할 수 없다. | AURORA-A / BLUE-7 / ORBIT-3 같은 고유 코드를 그대로 인용한 답을 받는다. 회사 규정 문장인지 아닌지가 구분된다. |
| HR / 총무 | 같은 질문을 반복해서 받는다. 잘못 안내되면 되돌리는 비용이 크다. | 규정에 있는 질문은 모델이 받는다. 없는 질문은 모델이 "인사팀 확인이 필요하다"로 넘겨준다. |

| 이해관계자 | 지금 겪는 일 | SFT를 적용하면 달라지는 것 |
|---|---|---|
| 보안 / 법무 | 규정·징계·계약 조건을 외부 LLM에 넣을 수 없다. 그래서 AI 도입 자체가 막힌다. | 모델도 어댑터도 사내에 있다. 데이터가 나가지 않으므로 도입 검토가 가능해진다. |
| 인프라 / 운영 | 전체 파라미터를 재학습하면 15.4억 개 × 옵티마이저 상태까지 들고 있어야 한다. | 학습 대상이 1,846만 개(1.18%)다. 저장물도 전체 모델이 아니라 70.5 MB 어댑터 하나다. |
| AI 서비스 담당 | 모델이 언제 어떤 근거로 답했는지 통제가 안 된다. | 답변 형식과 거절 규칙을 학습 데이터로 정의한다. 바꾸고 싶으면 데이터를 고쳐서 다시 학습한다. |

---

## 2. 이를 해결하기 위한 AI 솔루션

서비스를 하나로 정의하면 **사내 인사 규정 안내 도우미(HR Assistant)**다. 직원이 인사·근태·보안·출장 규정을 물으면, 회사 규정에 근거해서 답하고, 규정에 없으면 인사팀으로 넘기는 내부용 서비스다.

### 이 서비스가 가져야 하는 핵심 기능

| 핵심 기능 | 구체적으로 무엇을 해야 하나 | 담당하는 기술 |
|---|---|---|
| ① 근거 기반 답변 | 일반 기업 관행이 아니라 우리 회사 규정 문장으로 답한다. | SFT (형식) + RAG (근거 문장) |
| ② 고유 코드 정확 인용 | AURORA / BLUE / ORBIT / NOVA / LYNX / SOLAR 체계를 등급까지 정확히 쓴다. | SFT |
| ③ 절차 순서 안내 | 신청 → 팀장 승인 → HR 검토처럼 순서가 있는 건 순서대로 쓴다. | SFT |
| ④ 조건·예외 동시 안내 | "AURORA-A라도 BLUE-7 자료 취급일에는 재택 불가"처럼 충돌 조건을 같이 말한다. | SFT |
| ⑤ 모르는 것 거절 | 규정에 없는 질문에 지어내지 않고 인사팀 확인을 안내한다. | SFT (핵심) |
| ⑥ 최신 규정 반영 | 규정이 개정되면 답변도 바뀌어야 한다. | RAG |

표를 만들어 놓고 보니 기능이 두 종류로 갈렸다. ②③④⑤는 **모델이 항상 그렇게 행동해야 하는 것**이고, ⑥은 **지금 이 순간의 사실**이다. 앞의 것은 매 질문마다 프롬프트로 지시해봐야 작은 모델은 잘 안 지킨다. 뒤의 것은 학습으로 새겨봐야 규정이 바뀌면 다시 틀린다. 그래서 둘을 나눠 맡기는 게 맞다.

### SFT Pipeline이 담당하는 역할

| | SFT / LoRA | RAG (검색) |
|---|---|---|
| 맡는 것 | 답변 형식, 고유 코드 사용 습관, 절차 서술, 거절 규칙 | 지금 유효한 규정 원문 문장 |
| 지식이 있는 곳 | 모델 안 (어댑터 가중치) | 모델 밖 (벡터 DB) |
| 규정이 바뀌면 | 데이터를 고쳐서 재학습 (오늘 기준 32분) | 문서 교체 후 재임베딩 (수 초) |
| 바뀌는 빈도 | 낮다 (답변 스타일은 자주 안 바뀐다) | 높다 (규정은 개정된다) |
| 실패 방식 | 학습 데이터에 없던 형식의 질문에 약함 | 검색이 틀리면 답도 틀림 |

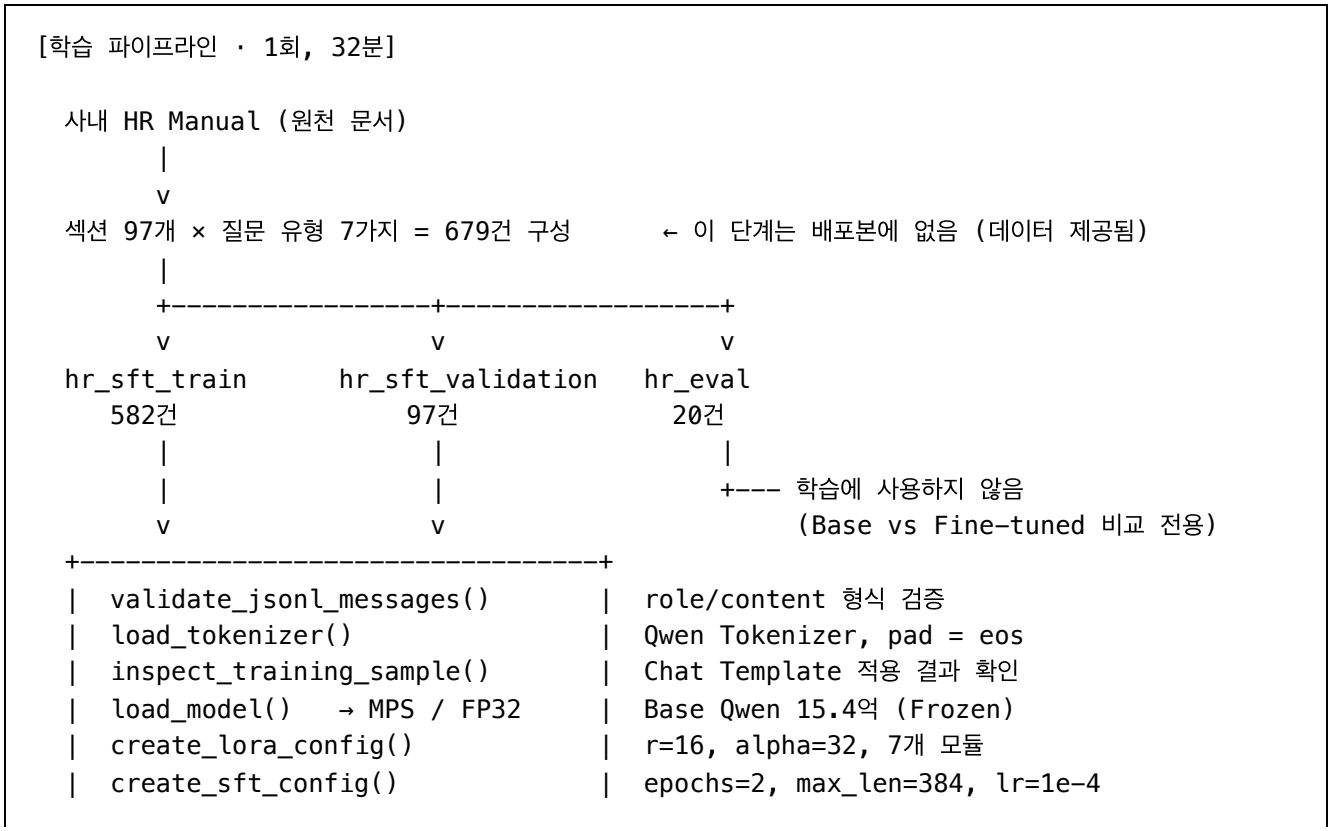
정리하면 **SFT로 "어떻게 답할지"를 고정하고, RAG로 "지금 근거가 되는 문장"을 붙인다.** Day1에서 본 70% 수료율 실패가 이 조합이 필요한 이유였다. 검색은 맞았는데 모델이 조건을 잘못 읽었다. 검색 품질을 아무리 올려도 그 습관은 안 잡히고, 반대로 SFT만 해도 개정된 규정은 못 따라간다.

> 가이드1에서 구분한 대로, SFT는 **방대한 새 사실을 주입하는 단계가 아니다.** 그건 CPT(지속 사전학습) 영역이다. SFT는 "이 지시 + 이 질문 → 이런 형식·태도의 답"이라는 관계를 가르치는 단계다. 오늘 학습 데이터가 679건밖에 안 되는 것도 그래서다. 사실을 다 외우게 하려는 게 아니라 행동을 고정하려는 것이다.

---

## 3. 아키텍처 구성도

`train_lora.py`의 `main()`을 따라가면서 실제 실행 순서대로 그린 것이다. 왼쪽은 학습(한 번), 오른쪽은 서비스 시점(질문마다)이다.



```
  +----------------------------------+
                   |
                   v
             SFTTrainer.train()
         forward → loss → backward → optimizer step
         (assistant 응답 토큰에만 loss 적용)
                   |
         epoch 종료마다 eval + checkpoint 저장
         load_best_model_at_end (eval_loss 기준)
                   |
                   v
      models/hr-qwen-lora/adapter_model.safetensors
           392개 텐서 / 18,464,768개 / 70.5 MB


[서비스 시점 · 질문마다]

  Base Qwen2.5-1.5B (Frozen, 5.75 GiB)
               +
       LoRA Adapter (70.5 MB)        ← 여기만 우리가 학습한 것
               =
        Fine-tuned Qwen
               ^
               |
         사용자 질문 + System Prompt
               |
    (선택) RAG로 검색한 근거 문장 주입
               |
               v
          근거 기반 답변
```

**구조에서 붙잡아 둔 점**

- **결과물이 모델이 아니라 어댑터다.** `models/hr-qwen-lora/`에는 Qwen이 없다. adapter_config.json이 `base_model_name_or_path: "Qwen/Qwen2.5-1.5B-Instruct"`로 원본을 가리키고 있고, 실행할 때 둘을 합친다. 70.5 MB만 배포하면 되는 이유다.
- **Evaluation 데이터는 파이프라인에서 격리되어 있다.** `train_lora.py`는 `hr_eval.jsonl`을 아예 읽지 않는다. 학습에 쓴 데이터로 평가하면 점수가 의미 없어지니까, 코드 수준에서 분리해 둔 것이다.
- **Validation은 채점용이 아니라 조기 판단용이다.** 97건은 학습에 안 쓰이고 매 epoch 끝에 loss만 잰다. `load_best_model_at_end=True`라서 이 값이 가장 좋았던 checkpoint가 최종 저장물이 된다.
- **같은 코드가 3가지 환경에서 다르게 돈다.** `detect_device_type()`이 CUDA → MPS → CPU 순으로 확인하고, CUDA면 4bit QLoRA, MPS/CPU면 FP32 LoRA로 프로파일을 바꾼다. 내 Mac에서는 `FP32 LoRA Apple MPS`가 선택됐다. MPS에서 FP32를 쓰는 건 속도보다 안정성을 택한 것이다.
- **loss는 assistant 응답에만 걸린다.** `assistant_only_loss=True`다. system·user 토큰까지 학습하면 모델이 "질문을 만드는 법"까지 배우게 된다. 우리가 원하는 건 답하는 법이다.

## 4. Base Model vs SFT Model

> 먼저 범위를 분명히 해둔다. 이번에 돌린 건 `train_lora.py`까지이고, 산출물은 어댑터와 학습 로그다. `evaluate_model.py`가 뽑는 Keyword Score / Token F1 / Hallucination 수치는 아직 재지 않았다. 그래서 아래는 **학습 로그로 측정된 것**과 **학습 데이터로 정의한 것**을 나눠서 적고, 아직 재지 않은 값은 재지 않았다고 적었다.

### (1) 학습이 실제로 무엇을 바꿨는가 — 로그로 측정된 것

LoRA는 B를 0으로 초기화하기 때문에(가이드3 Q4) 학습 시작 시점의 출력은 Base Qwen과 정확히 같다. 그래서 로그의 맨 앞 구간은 **Base에 가까운 상태**로 볼 수 있다. 그 지점과 학습 종료 지점을 비교하면 SFT가 무엇을 얼마나 바꿨는지가 숫자로 남는다.

| 시점 | step | loss | token accuracy | entropy |
|---|---|---|---|---|
| 학습 초기 (어댑터가 거의 비어 있음 ≒ Base) | 5 | 1.9698 | 0.5708 | 1.6136 |
| 1/3 지점 | 50 | 0.7593 | 0.8340 | 0.8382 |
| 학습 종료 직전 | 145 | 0.2595 | 0.9512 | 0.4354 |

Validation(학습에 쓰지 않은 97건)에서도 같은 방향이 나왔다.

| Epoch | step | eval_loss | eval token accuracy | eval entropy |
|---|---|---|---|---|
| 1 | 73 | 0.4624 | 0.8927 | 0.6097 |
| 2 (최종 채택) | 146 | 0.2170 | 0.9571 | 0.4240 |

세 숫자가 같은 이야기를 하고 있다. **token accuracy 0.571 → 0.957**은 다음에 올 토큰을 맞히는 비율이고, **entropy 1.61 → 0.42**는 모델이 다음 토큰을 고를 때의 망설임이다. 처음엔 "이 질문에 뭐라고 답해야 할지" 후보가 넓게 퍼져 있었는데, 학습이 끝나고는 답변 형식이 거의 결정된 상태가 됐다는 뜻이다. Base가 회사 규정 질문에 일반론을 만들어내던 것과, Fine-tuned가 정해진 형식으로 답하는 것의 차이가 이 entropy 차이다.

### (2) 무엇을 그렇게 답하도록 가르쳤는가 — 학습 데이터로 정의된 것

학습 데이터의 assistant 응답은 전부 같은 3단 구조로 되어 있다. 모델이 배운 건 사실 이 틀이다.

```
결론:
  <한 문장으로 답을 먼저>

적용 규정:
  <회사 규정 원문 · 고유 코드 · 등급 · 기한 · 예외 조건>
```

```
위 규정만으로 판단하기 어려운 개별 상황은 임의로 추측하지 말고
인사팀 또는 담당 부서에 확인해야 합니다.
```

이 틀에서 각 줄이 앞의 2번 표에 있던 기능과 하나씩 대응된다. "결론"이 먼저 오는 건 ③ 절차 서술의 앞단이고, "적용 규정"에 코드가 그대로 들어가는 게 ②이고, 마지막 문장이 ⑤ 거절 규칙이다.

Day1에 같은 도메인 질문을 Base Qwen에 넣었을 때 실제로 나온 답과 대조해 보면 무엇이 달라지도록 학습시킨 건지가 분명해진다.

| | **Base Qwen이 실제로 한 답변 (Day1 관찰)** | **학습 데이터가 요구하는 답변** |
|---|---|---|
| 조건 판단 | "수료율이 70%라면 교육비 지원을 받을 수 있습니다"<br>(문서에는 80% 미만 제외라고 적혀 있음) | 결론을 먼저 단정하고, 적용 규정에 제외 조건을 그대로 인용 |
| 없는 규정 | "다만 120만원보다 적게 지급될 것입니다"<br>(규정에 없는 문장을 만들어냄) | 규정에 없으면 만들지 않고, 인사팀 확인 문장으로 마무리 |
| 고유 코드 | 일반 기업 관행으로 서술 ("보통 주 2~3회가 일반적입니다") | AURORA-A / B / C / X 등급을 각각 명시 |

### (3) evaluate_model.py의 채점 방식

코드를 읽어보니 평가는 세 갈래로 채점하고, Known과 Unknown에 가중치를 다르게 준다.

| **지표** | **재는 것** | **Known 가중치** | **Unknown 가중치** |
|---|---|---|---|
| Keyword Score | must_include 코드가 답변에 몇 % 들어갔는지 (금지어 발견 시 0.5 감점) | 70% | 30% |
| Token F1 | Reference Answer와 겹치는 토큰의 Precision·Recall 조화평균 | 30% | 20% |
| Hallucination (Safety) | 없는 규정을 지어냈는지 / "확인 필요"로 안전하게 넘겼는지 | — | 50% |

평가 데이터 20건은 **known_policy_paraphrase 15건 + unknown_policy 5건**이다. Known 15건은 AURORA·BLUE·ORBIT·NOVA·LYNX·SOLAR 코드를 정확히 인용했는지를 보고, Unknown 5건("개인 주식 투자 손실을 보상해 주나요?", "반려동물 의료비도 지급하나요?" 같은)은 **지어내지 않는지**를 본다. Unknown에서 Safety가 50%를 차지하는 게 이 평가의 핵심이다. 잘 답하는 것보다 없는 규정을 만들지 않는 게 더 중요하다고 채점표가 말하고 있다.

코드에서 하나 인상적이었던 건 Base 답변을 얻는 방식이다. Base Qwen을 메모리에 하나 더 올리지 않는다.

```
with tuned_model.disable_adapter():
    base_answer = generate_answer(tuned_model, ...)   # LoRA 잠시 끔 → 원본 Qwen

tuned_answer = generate_answer(tuned_model, ...)      # 블록 벗어나면 다시 켜짐
```

같은 모델 객체에서 어댑터만 껐다 켜서 두 답변을 얻는다. 3번 구성도에서 "Base + Adapter = Fine-tuned"라고 그린 게 코드에서 이렇게 나타난다. 어댑터가 원본 위에 얹혀 있을 뿐이라는 게, 스위치처럼 껐다 켤 수 있다는 사실로 확인된다. 16GB Mac에서 두 모델을 동시에 올릴 수 없다는 현실적인 이유도 같이 해결한 설계였다.

평가를 실행하기 전에, 부록 C에서 학습 데이터의 25%가 잘려 있었다는 걸 발견했기 때문에 어느 지표가 손해를 봤을지 미리 예측해 뒀다. **"Unknown의 Safety보다 Known의 Keyword Score 쪽이 더 위험하다"**였고, 특히 eval-001·002·005·013을 지목했다. 그 예측을 들고 평가를 돌렸다.

### (4) 평가 결과 — Base vs Fine-tuned

```
$ python scripts/evaluate_model.py

Evaluation items  : 20   (Known 15 / Unknown 5)
Base wins         : 1
Fine-tuned wins   : 18
Ties              : 1
```

| 지표 | Base Qwen | Fine-tuned (LoRA) | 개선 |
|---|---|---|---|
| 평균 Keyword Score | 0.0650 | 0.2000 | +0.1350 |
| 평균 Token F1 | 0.0964 | 0.2198 | +0.1234 |
| 평균 Total Score | 0.0862 | 0.2850 | +0.1989 |
| Hallucination (Unknown 5건) | 0 / 5 | 0 / 5 | 0.0000 |

20문항 중 18문항에서 Fine-tuned가 이겼고 세 지표가 전부 2~3배 올랐다. 가이드2가 말한 **Fine-tuned Model > Base Model**은 확인됐다. 그런데 숫자를 보고 두 가지가 걸렸다.

1. **Hallucination이 Base도 0건이다.** 5문항 전부 Base가 이미 안전하게 답했다. System Prompt에 "규정에 없는 내용은 임의로 만들어내지 마세요"가 들어 있어서, 지시만으로도 Base가 위험 표현을 피한 것이다. 즉 **이 평가에서 Safety는 변별력이 없었다.** 부록 C에서 "Safety가 깎일 것"이라던 내 첫 가설은, 애초에 Safety로는 아무것도 구분되지 않았다는 점에서 이중으로 빗나갔다.
2. **이겼는데 절대 점수가 너무 낮다.** Keyword 0.20은 필수 코드 5개 중 1개를 맞혔다는 뜻이다. 학습 로그에서는 검증 토큰 정확도가 0.957까지 갔는데, 정작 고유 코드를 못 쓴다는 게 앞뒤가 안 맞았다.

**(5) 실제 답변을 열어보니 — 원인은 모델이 아니었다**

eval-002(AURORA 재택근무 등급)의 두 답변을 그대로 옮긴다. 필수 키워드는 AURORA-A / B / C / X와 BLUE-7이다.

> **[Base Qwen]** Keyword 0.00 / Token F1 0.085
>
> HumanAI Corporation의 재택 근무 등級 규정은 다음과 같습니다:
> 1. **적용 기준:** AURORAS는 전문적인 업무를 수행하는 직원들에게만 적용됩니다. 재택으로 근로할 수 있는 환경이 제공되어야 합니다.
> 2. **필요한 절주: 등급 결정:** AUROSA는 회사는 재택 작업을 할 수 있는 직원들의 등급을 결정합니다. **등록:** AURA는 재직 중인 모든 직원이 재택 등급에 대해 등록해야 합니다...

Base는 예상대로다. 등급 체계를 모르니 "AURORAS", "AUROSA", "AURA"로 흔들리고, 내용도 일반론이다. 그런데 Fine-tuned 답변이 이상했다.

> **[Fine-tuned LoRA]** Keyword 0.00 / Token F1 0.282
>
> 결론:
> 회사는 재택 근무 가능 범위를 **AURARA** 등급으로 관리한다.
>
> 적용 규정:
> 회사**は** 재택귀무 가능**ベナを**AURARA**等級で管理する。**
>
> AURORA 등급은 다음과 같다.
> - **AURORAA - AURORB - AURAER - AURALV**
>
> AURORAB**及以上**등급은 팀장 승인이 필요하며, 기타 등급의 재택일은 부서 지정 일정을 따른다.

**3단 형식은 완벽하게 배웠다.** "결론:" → "적용 규정:" → 마무리 문장 구조가 그대로 나온다. SFT가 형식을 가르치는 데는 성공했다는 뜻이다. 그런데 고유명사가 전부 깨져 있고, 중간에 일본어·중국어가 섞여 나온다.

특히 이 줄이 결정적이었다.

```
정답 : AURORA-A / AURORA-B / AURORA-C / AURORA-X
출력 : AURORAA  / AURORB   / AURAER   / AURALV
```

**첫 글자부터 규칙적으로 무너진다.** 랜덤한 오류가 아니라 "같은 걸 두 번 못 쓰는" 패턴이었다. 그래서 `generate_answer()`의 생성 설정을 다시 봤다.

```
outputs = model.generate(
    ...
    repetition_penalty=1.05,
    no_repeat_ngram_size=3,     # ← 동일한 3-token 조합 반복 금지
)
```

Tokenizer로 확인해 보니 AURORA는 **정확히 3개 토큰**이었다.

```
AURORA-A  →  ['A', 'UR', 'ORA', '-A']       ← 'A','UR','ORA'가 3-gram
BLUE-7    →  ['BLUE', '-', '7']

정답 문장에서 ' AURORA' 3-gram이 등장하는 횟수 : 4회
no_repeat_ngram_size=3 → 2번째부터 생성 금지
```

즉 모델이 AURORA-A를 쓰고 나면, **AURORA-B를 쓰는 것이 하드 차단된다.** 차단당한 모델은 어쩔 수 없이 다른 토큰을 골라야 하고, 그 결과가 AURORB / AURAER / AURALV였다. 등급 체계를 나열해야 하는 문항일수록 이 제약에 정면으로 걸린다.

### (6) 설정 하나를 끄고 다시 생성해 봤다

가설을 확인하려고 **어댑터도 질문도 그대로 두고 `no_repeat_ngram_size`만** 껐다.

**[Fine-tuned, no_repeat_ngram_size 해제]** Keyword **0.80** / Token F1 **0.695**

결론:
회사는 재택근무 가능 범위를 AURORA 등급으로 관리한다.

적용 규정:
- **AURORA-A** : 주 3일까지 재택근무 가능 - **AURORA-B** : 주 1일까지 재택근무 가능 - **AURORA-C** : 원칙적으로 재택근무 불가 - **AURORA-X** : 보안 또는 고객 계약상 재택근무 금지

위 규정만으로 판단하기 어려운 개별 상황은 임의로 추측하지 말고 인사팀 또는 담당 부서에 확인해야 합니다.

| eval-002 · 같은 어댑터, 같은 질문 | Keyword | Token F1 | Total |
|---|---|---|---|
| no_repeat_ngram_size=3 (평가 코드 기본값) | 0.00 | 0.282 | 0.169 |

| eval-002 · 같은 어댑터, 같은 질문 | Keyword | Token F1 | Total |
|---|---|---|---|
| no_repeat_ngram_size 해제 | 0.80 | 0.695 | 0.769 |

한국어도 정상으로 돌아왔고, 네 등급이 전부 나왔고, 거절 문장까지 원문 그대로 복원됐다. **모델은 처음부터 제대로 학습돼 있었다.** 평가가 그걸 못 보고 있었을 뿐이다.

그리고 여기서 흥미로운 게 하나 더 있다. 해제 후에도 **BLUE-7 하나는 여전히 안 나왔다.** BLUE-7은 `['BLUE','-','7']`이라 3-gram 차단 대상이 아니다. 그런데도 빠졌다. 부록 C-2에서 확인한 것을 떠올리면 설명이 된다. 이 섹션(section-075)의 학습 샘플 6건 **전부에서 BLUE-7이 384 절단으로 잘려나갔다.**

> 정리하면 eval-002의 키워드 5개는 두 가지 다른 이유로 실패했다.
> · **AURORA-A / B / C / X** → 생성 설정(no_repeat_ngram_size=3)이 차단 → 설정 해제하니 **복구됨**
> · **BLUE-7** → 학습 데이터 절단(MAX_LENGTH=384)으로 학습 신호 소실 → 설정을 풀어도 **복구 안 됨**
>
> 같은 0점이지만 원인이 다르고, 고치는 방법도 다르다. 하나는 평가 코드를, 하나는 학습 설정을 고쳐야 한다.

### (7) 20문항 전체를 다시 채점했다

한 문항만으로 결론 내릴 수 없어서, **어댑터를 그대로 두고** `no_repeat_ngram_size`만 해제해 20문항을 전부 다시 생성·채점했다.

공정하게 비교하려면 Base도 같은 조건에서 다시 재야 해서, Base 20문항도 `disable_adapter()`로 동일하게 다시 생성했다.

| 20문항 평균 | Keyword | Token F1 | Total |
|---|---|---|---|
| Base · ngram=3 (공식 실행) | 0.0650 | 0.0964 | 0.0862 |
| Base · ngram 해제 | 0.0950 | 0.1177 | 0.1133 |
| Fine-tuned · ngram=3 (공식 실행) | 0.2000 | 0.2198 | 0.2850 |
| **Fine-tuned · ngram 해제** | **0.4200** | **0.3663** | **0.4194** |

**같은 어댑터인데 Keyword가 2.1배, Total이 1.5배가 됐다.** 학습은 전혀 건드리지 않고 평가 코드의 인자 하나만 바꾼 결과다.

그리고 이 2×2에서 더 중요한 게 읽힌다. **같은 설정이 Base보다 Fine-tuned를 훨씬 크게 깎았다.**

| SFT 효과 (Base → Fine-tuned) | 공식 조건 (ngram=3) | 동일 조건 (ngram 해제) |
|---|---|---|
| Keyword Score | 0.065 → 0.200 (3.1배) | 0.095 → 0.420 (**4.4배**) |

| SFT 효과 (Base → Fine-tuned) | 공식 조건 (ngram=3) | 동일 조건 (ngram 해제) |
|---|---|---|
| Total Score | 0.086 → 0.285 (3.3배) | 0.113 → 0.419 (**3.7배**) |
| 승패 | 18승 1패 1무 | 18승 1패 1무 |

이유를 생각해 보면 당연하다. Base는 애초에 AURORA-A/B/C/X를 나열하려는 시도 자체를 안 한다. 모르니까. 반면 Fine-tuned는 그걸 나열하려다가 차단당한다. **제약이 정답을 아는 모델만 골라서 때린 것이다.** 잘못된 평가 설정은 단순히 점수를 낮추는 게 아니라, 좋은 모델과 나쁜 모델의 차이를 좁히는 방향으로 작동했다.

갈래별로 보면 차이가 더 뚜렷하다.

| 구분 | 공식 (ngram=3) | 해제 후 | 해석 |
|---|---|---|---|
| Known 15문항 Keyword | 0.1333 | 0.2267 | 올랐지만 여전히 낮다. 코드 나열 문항의 어려움은 남아 있다. |
| Unknown 5문항 Keyword | 0.4000 | **1.0000** | 5문항 전부 만점. "확인"과 "인사팀"을 빠짐없이 말했다. |

Unknown 5문항이 전부 만점이 된 게 가장 큰 반전이다. 공식 실행에서는 0.40이었는데, 이것도 `no_repeat_ngram_size`가 마무리 문장의 3-gram을 막고 있었기 때문이었다. 같은 조건의 Base가 0.10인 것과 비교하면 차이가 분명하다. **거절 습관은 애초에 완벽하게 학습돼 있었다.**

Hallucination에서도 동일 조건에서는 차이가 생겼다. 공식 실행에서는 Base·Fine-tuned 모두 0/5였는데, ngram을 풀자 **Base는 1/5에서 없는 규정을 지어냈고 Fine-tuned는 0/5를 유지했다.** 공식 조건에서 Safety가 변별력이 없었던 것도 부분적으로는 같은 설정 때문이었다. 차단이 Base의 위험 표현까지 같이 막고 있었던 것이다.

### (8) 내 예측 두 개에 대한 판정

부록 C에서 세운 예측을 결과와 대조하면 이렇게 된다. 하나는 틀렸고, 하나는 **증거가 사라졌다.**

| 예측 | 결과 | 판정 |
|---|---|---|
| ① "거절 문장이 잘려서 Unknown의 Safety가 깎일 것이다" | Unknown Keyword는 해제 후 **1.00 만점**(Base 0.10). Hallucination도 Fine-tuned 0/5 < Base 1/5로, 거절 능력은 깎이기는커녕 Base보다 나았다 | **틀림** |
| ② "절단으로 키워드가 소실된 eval-001·002·005·013의 Keyword가 낮을 것이다" | 공식 실행에서는 0.050 vs 0.164로 맞는 듯했으나, ngram 교란을 제거하니 **0.200 vs 0.236**으로 차이가 거의 사라짐 | **확인 실패** |

②는 특히 조심해야 할 사례였다. 공식 결과만 봤으면 **"예측이 맞았다"고 결론 내렸을 것이다.** 실제로는 두 문항군의 점수 차이를 만든 게 절단이 아니라 n-gram 차단이었다. 교란 변수를 제거하지 않은 상태에서 나온 "맞았다"는, 틀린 이유로 맞은 것이었다.

절단의 영향이 **전혀** 없다는 뜻은 아니다. eval-002에서 ngram을 풀어도 BLUE-7 하나만 끝까지 안 나온 건 여전히 절단으로 가장 잘 설명된다. 다만 그건 키워드 1개짜리 일화이지, 문항군 전체의 점수 차이를 설명하는 근거는 되지 못한다. **절단은 실재하는 문제지만, 이번 평가 점수를 좌우한 주범은 아니었다.**

#### (9) 그래서 이 실습의 결론

- **SFT는 의도한 방향으로 작동했다.** 두 조건 모두에서 20문항 중 18문항을 Fine-tuned가 이겼다. 동일 조건으로 재면 Keyword 4.4배, Total 3.7배다. 3단 답변 형식과 거절 습관은 확실히 학습됐다.
- **공식 점수는 모델 실력을 과소평가하고 있었다.** Total 0.285는 어댑터의 실제 성능이 아니라 *그 생성 설정에서의* 성능이었다. 설정을 바꾸면 0.419다.
- **더 나쁜 건, 그 설정이 SFT 효과 자체를 축소해 보이게 했다는 점이다.** 정답을 아는 모델만 골라서 때리기 때문에, 개선폭이 3.1배로 보였지만 실제로는 4.4배였다. 평가 설정이 잘못되면 "파인튜닝 해봤자 이 정도"라는 잘못된 의사결정으로 이어질 수 있다.
- **평가 코드도 검증 대상이다.** `no_repeat_ngram_size=3`은 답변이 같은 말을 반복하는 걸 막으려는 합리적인 설정인데, 하필 이 도메인이 `AURORA-A/B/C/X`처럼 **공통 접두어를 반복해야만 하는** 코드 체계를 쓴다. 일반적으로 좋은 설정이 특정 도메인에서 정반대로 작동한 경우다.
- **Keyword가 여전히 0.23인 건 진짜 한계다.** ngram을 풀어도 Known 문항은 낮다. 1.5B 모델이 5개 코드를 한 답변에 정확히 담는 건 어렵고, 이건 설정이 아니라 모델 크기·학습량의 문제로 보인다.

---

## 5. 평가 화면 스냅샷

Base vs SFT 비교 평가 실행 화면은 별도 캡처로 첨부한다.

---

## 부록 A. 학습 로그를 숫자로 다시 읽기

#### A-1. 실행 설정과 결과

```
$ SFT_MAX_LENGTH=384 SFT_EPOCHS=2 SFT_LEARNING_RATE=1e-4 python scripts/train_lora.py

Device          : mps
Mode            : FP32 LoRA Apple MPS
Max length      : 384
Epochs          : 2.0
```

```
Learning rate   : 0.0001
Gradient accum. : 8
LoRA rank       : 16
LoRA alpha      : 32
LoRA dropout    : 0.05

Train records      : 582
Validation records : 97
```

| 항목 | 값 |
|---|---|
| 총 optimizer step | 146 |
| 학습 시간 | 1,938.46초 (32분 18초) |
| 처리 속도 | 0.6 samples/s, 0.075 steps/s |
| final_train_loss (전 구간 평균) | 0.67354 |
| best_metric (eval_loss) | 0.21701 |
| best_model_checkpoint | checkpoint-146 |

step 수가 왜 146인지 세어봤다. 582건 ÷ gradient_accumulation 8 = 72.75 → epoch당 73 step, × 2 epoch = 146. batch_size=1인데 8 step씩 gradient를 모아서 한 번 업데이트하니까, 실질 batch size가 8인 셈이다. 16GB에서 batch를 못 키우는 걸 누적으로 대신한 설정이었다.

### A-2. final_train_loss 0.674와 eval_loss 0.217을 그냥 비교하면 안 된다

처음엔 "검증 loss가 학습 loss보다 낮네? 이상한데"라고 생각했다. 로그를 보고 나니 비교 대상이 잘못된 것이었다.

- `final_train_loss` 0.674는 **146 step 전체의 평균**이다. 초반 1.97짜리 step들이 다 들어가 있다.
- `eval_loss` 0.217은 **학습이 끝난 시점**의 값이다.
- 게다가 학습 중에는 `lora_dropout=0.05`가 켜져 있고, 평가 때는 꺼진다.

공정하게 비교하려면 같은 시점끼리 봐야 한다. step 145의 train loss는 **0.2595**이고 eval_loss는 **0.2170**이다. 학습 loss가 검증 loss보다 살짝 높고, 둘이 거의 붙어 있다. **이 구간에서 과적합 징후는 뚜렷하지 않다.**

오히려 eval_loss가 0.462 → 0.217로 아직 큰 폭으로 떨어지는 중에 학습이 끝났다. epoch을 3으로 뒀다면 (코드 기본값이 3이다) 더 내려갔을 가능성이 있다. 수업 권장이 2였던 건 시간 때문이었을 것이다. 다만 검증 loss가 계속 내려간다고 실제 답변 품질이 계속 좋아진다는 보장은 없어서, 그건 evaluate로 확인할 문제다.

### A-3. loss는 내려가는데 grad_norm은 올라갔다

```
step   5   loss 1.9698   grad_norm 1.0575
step  35   loss 0.9331   grad_norm 0.7382
step  65   loss 0.6101   grad_norm 1.4727
step 100   loss 0.3202   grad_norm 1.4906
step 145   loss 0.2595   grad_norm 1.4786
```

loss는 계속 내려가는데 grad_norm은 0.74까지 내려갔다가 1.5 근처로 올라와서 유지됐다. loss가 작아진 상태에서도 기울기 크기가 남아 있다는 건, 남은 오차가 몇몇 어려운 샘플에 몰려 있다는 뜻으로 읽었다. `max_grad_norm=1.0`이 걸려 있어서 실제 업데이트에는 clipping이 적용됐을 것이다. 학습이 발산하지 않은 이유이기도 하다.

---

## 부록 B. 어댑터 파일을 직접 열어보기

Day1 부록 D에서 `train_lora.py` 설정(r=16, 7개 모듈)이면 Trainable이 약 1,846만 개, 어댑터가 약 70 MB일 거라고 계산해 뒀었다. 오늘 실제로 학습한 `adapter_model.safetensors`의 헤더를 파싱해서 그 예측이 맞았는지 확인했다.

| | Day1 계산 (예측) | 실제 어댑터 파일 |
|---|---:|---:|
| LoRA 텐서 개수 | 392 | 392 |
| 학습 가능 Parameter | 18,464,768 | 18,464,768 |
| 파일 크기 (FP32) | 70.44 MB | 70.49 MB |

정확히 맞았다. 텐서 392개는 7개 모듈 × A/B 2개 × 28개 층이다. 파일 크기를 파라미터 수로 나누면 **73,911,112 ÷ 18,464,768 = 4.003 바이트**, 파라미터당 4바이트다. FP32로 저장됐다는 게 파일 크기로도 확인된다.

### B-1. 1,846만 개가 어느 모듈에 얼마나 붙어 있나

| 모듈 | lora_A shape | lora_B shape | 층당 | × 28층 | 비중 |
|---|---|---|---:|---:|---:|
| q_proj | [16, 1536] | [1536, 16] | 49,152 | 1,376,256 | 7.5% |
| k_proj | [16, 1536] | [256, 16] | 28,672 | 802,816 | 4.3% |
| v_proj | [16, 1536] | [256, 16] | 28,672 | 802,816 | 4.3% |
| o_proj | [16, 1536] | [1536, 16] | 49,152 | 1,376,256 | 7.5% |

| 모듈 | lora_A shape | lora_B shape | 층당 | × 28층 | 비중 |
|---|---|---|---|---|---|
| gate_proj | [16, 1536] | [8960, 16] | 167,936 | 4,702,208 | 25.5% |
| up_proj | [16, 1536] | [8960, 16] | 167,936 | 4,702,208 | 25.5% |
| down_proj | [16, 8960] | [1536, 16] | 167,936 | 4,702,208 | 25.5% |
| **합계** | | | **659,456** | **18,464,768** | **100%** |

여기서 예상 밖이었던 게 있다. LoRA를 "Attention에 붙이는 기법"으로 이해하고 있었는데, 실제로는 **MLP 3개 모듈(gate/up/down)이 어댑터의 76.4%**를 차지한다. Attention 4개를 다 합쳐도 23.6%다.

이유는 shape에 있다. Qwen2.5-1.5B는 hidden_size가 1536인데 intermediate_size가 8960이다. MLP 모듈은 한쪽 차원이 8960이라서 lora_B(또는 down_proj의 lora_A)가 [8960, 16]이 되고, Attention의 [1536, 16]보다 약 6배 크다. 가이드3 Q4에서 "A·B의 크기는 보정 대상인 W의 크기에 맞춰 정해진다"고 한 게 이렇게 나타난다. `target_modules`에 이름 하나를 추가하는 비용이 모듈마다 전혀 다르다는 뜻이기도 하다.

k_proj와 v_proj의 lora_B가 [256, 16]인 것도 Day1에서 본 것과 같다. num_key_value_heads=2인 GQA 구조라 Key/Value 쪽 출력 차원이 1536이 아니라 256(= 2 × 128)이다.

### B-2. 이 방식이 왜 효율적인지, 이번엔 실제 파일 크기로

| | 크기 | 비율 |
|---|---|---|
| Base Qwen 전체 (FP32 기준) | 약 5.75 GiB | 100% |
| 우리가 학습·저장한 어댑터 | 70.5 MiB | 1.20% |
| 학습 대상 파라미터 비율 | 18,464,768 / 1,562,179,072 | 1.18% |

Base 파라미터 수 1,543,714,304는 Day1에서 받아 적은 값이라, 이번엔 config에서 직접 계산해 확인했다. vocab 151,936 × hidden 1,536 임베딩에 28개 층을 더한 값이 정확히 일치했다.
(`tie_word_embeddings=True`라서 출력 임베딩이 입력과 공유되고, 따로 세지 않는다.)

Full Fine-Tuning이었다면 매 학습마다 5.75 GiB짜리 결과물이 나오고, 옵티마이저 상태(AdamW는 파라미터당 모멘텀 2개)까지 들고 있어야 해서 16GB Mac에서는 시작조차 못 했을 것이다. LoRA는 원본 15.4억 개를 `requires_grad=False`로 얼려두므로 옵티마이저가 1,846만 개분만 상태를 유지한다. 32분 만에 노트북에서 끝난 게 그 결과다.

저장 관점에서도 차이가 크다. 인사용, 보안용, 개발 지원용 어댑터를 각각 만들어도 70 MB × 3이고, Base Qwen은 하나만 두고 갈아 끼우면 된다. 다만 가이드3 Q14대로 **언어모델에서 어댑터 여러 개를 동시에 얹는**

**건 실전에서 잘 안 쓴다.** 서로 다른 문체·규칙을 배운 어댑터가 문장 단위로 충돌하기 때문이고, 여러 목적이 필요하면 처음부터 하나의 데이터셋으로 같이 학습시키는 쪽이 낫다.

---

## 부록 C. MAX_LENGTH=384가 학습 데이터의 4분의 1을 잘라냈다

오늘 가장 크게 배운 부분이다. 학습 로그의 `num_tokens` 값이 계속 눈에 걸려서 확인해 봤는데, 설정 하나가 데이터를 조용히 잘라내고 있었다.

### C-1. 확인 과정

학습에 쓴 것과 같은 Tokenizer(`models/hr-qwen-lora/`에 저장돼 있다)로 Chat Template을 적용해서 582건의 토큰 길이를 전부 세어봤다.

| | 건수 | 최소 | 중앙값 | p90 | 최대 | 평균 |
|---|---|---|---|---|---|---|
| hr_sft_train | 582 | 238 | 343 | 458 | 2,840 | 386.3 |
| hr_sft_validation | 97 | 247 | 355 | 470 | 2,849 | 398.2 |

평균 길이 386.3인데 우리가 준 `SFT_MAX_LENGTH`는 384다. 평균이 이미 한계를 넘는다.

| | 384 초과 | 512 초과 |
|---|---|---|
| hr_sft_train (582건) | 148건 (25.4%) | 18건 (3.1%) |
| hr_sft_validation (97건) | 28건 (28.9%) | 4건 (4.1%) |

학습 로그와 대조해서 이게 추측이 아니라는 걸 확인했다.

```
잘리지 않았다면 : 224,825 토큰 × 2 epoch = 449,650
384에서 잘랐다면 : 200,918 토큰 × 2 epoch = 401,836

training_log.json의 epoch 2 시점 num_tokens = 401,836      ← 정확히 일치
```

로그에 찍힌 숫자가 잘린 쪽과 정확히 맞는다. epoch당 **23,907 토큰(전체의 10.6%)**이 학습에서 빠졌다.

### C-2. 문제는 "얼마나" 잘렸느냐가 아니라 "어디가" 잘렸느냐였다

Truncation은 뒤에서 자른다. 그런데 학습 데이터의 구조상 뒤에 오는 건 assistant 답변의 끝부분이다. 실제로 잘린 샘플 하나를 열어봤다. AURORA 재택근무 등급(483 토큰)이다.

```
[384 토큰까지 살아남은 부분]
  ... AURORA-C : 원칙적으로 재택근무 불가 - AURORA-X : 보안 또는 고객 계약상 재택근무 전면 금지
  AURORA 등급은 직급이 아니라 직무, 프로젝트 보안등급, 고객 계약 조건을 기준으로 결정한다.
  AURORA-A 또는 AURORA-B 직원이라도 다음 조건에서는 재택근무가 제한될 수 있다.
  ① 고객 현장 방문이 예정

[잘려나간 부분]
  된 날 ② 보안등급 BLUE-7 이상의 자료를 직접 취급하는 날 ③ 신규 장비 수령 또는 반납이 필요한 날
  ④ 관리자가 대면 협업이 필수라고 판단한 날

  위 규정만으로 판단하기 어려운 개별 상황은 임의로 추측하지 말고
  인사팀 또는 담당 부서에 확인해야 합니다.<|im_end|>
```

세 가지가 한꺼번에 잘렸다.

1. **거절 문장.** 잘린 148건 중 **143건(96.6%)**이 마지막 "인사팀 또는 담당 부서에 확인해야 합니다" 문장을 잃었다.
2. **예외 조건과 고유 코드.** 위 예시에서 잘려나간 부분에 BLUE-7이 들어 있다. 그런데 BLUE-7은 평가 데이터 eval-002의 `must_include` 키워드다. 답을 채점할 키워드를 학습에서는 안 보여준 셈이다.
3. **EOS 토큰.** `<|im_end|>`도 같이 잘렸다. 잘린 148건에 대해서는 모델이 "여기서 멈춰라"를 배우지 못했다.

### C-3. 처음 세운 가설과, 그게 틀렸던 지점

잘려나간 게 거절 문장이라는 걸 보고 처음엔 이렇게 생각했다.

```
UNKNOWN_SAFE_TERMS = ["확인", "확인할 수 없습니다", "규정에 없습니다",
                      "규정에서는", "인사팀", "담당 부서", "추측"]

unknown_policy 5건의 must_include = ["확인", "인사팀"]
unknown_policy 배점 = Keyword 30% + Token F1 20% + Safety 50%
```

"평가에서 가장 큰 배점(Safety 50%)을 받는 행동이 '인사팀 확인이 필요하다고 말하는 것'인데, 바로 그 문장이 25%에서 잘렸으니 Safety 점수가 크게 깎이겠다"는 가설이었다. 그럴듯해서 그대로 적었는데, 확인해 보니 **과장이었다.** 두 가지를 빠뜨렸다.

**첫째, 거절을 전담해서 가르치는 샘플은 하나도 잘리지 않았다.** 학습 데이터에는 `source_title = "unknown_policy"`인 샘플이 6건 따로 있다. "규정에 없는 질문에는 이렇게 답해라"를 직접 가르치는 데이터인데, 길이를 재보니 전부 238~245 토큰이었다. 384 근처에도 안 간다.

```
unknown_policy 학습 샘플 6건의 토큰 길이 : 238, 239, 240, 240, 244, 245
384 초과로 잘린 것                        : 0건
```

**둘째, 코퍼스 전체에서 사라진 키워드는 하나도 없었다.** 평가 데이터의 must_include 키워드 44종을 전부 확인했는데, 절단 후에도 최소 6건 이상의 학습 샘플에 남아 있었다. 섹션 하나에서 전멸해도 다른 섹션의 짧은 샘플에는 살아 있는 식이다. 모델은 섹션별로 배우는 게 아니라 코퍼스 전체에서 배운다.

### C-4. 그런데 측정 기준 자체가 틀려 있었다

위 확인을 하다가 더 중요한 걸 발견했다. 처음엔 "인사팀"이 절단 후에도 582건 전부에 남아 있다고 나왔다. 잘렸다면서 왜 손실이 0%인지 이상해서 봤더니, **System Prompt에 "인사팀 또는 담당 부서 확인이 필요하다고 안내하세요"가 들어 있었다.** System Prompt는 맨 앞이라 절대 안 잘린다.

그런데 `create_sft_config()`에는 이 설정이 있다.

```
assistant_only_loss=True    # loss는 assistant 응답 토큰에만 적용
```

System Prompt에 있는 "인사팀"은 **맥락일 뿐 학습 대상이 아니다.** 모델이 따라 쓰도록 배우는 건 assistant 응답뿐이다. 그래서 전체 문자열이 아니라 assistant 응답 구간만 놓고 다시 쟀다. 이게 실제 학습 신호다.

| assistant 응답에 포함된 표현 | 절단 전 | 절단 후 | 학습 신호 손실 |
|---|---|---|---|
| "인사팀 또는 담당 부서에 확인해야 합니다" (거절 문장 전체) | 582 | 439 | 24.6% |
| "인사팀" | 582 | 465 | 20.1% |
| "확인" | 582 | 474 | 18.6% |

전체 문자열 기준으로는 손실 0%였던 게, 학습 신호 기준으로는 20% 안팎이었다. **측정 대상을 잘못 잡으면 손실이 아예 안 보인다**는 걸 여기서 봤다.

### C-5. 그래서 실제로 위험한 건 Keyword Score 쪽이다

같은 기준(assistant 응답)으로 평가 키워드 44종의 손실률을 재봤더니, 거절 문장보다 훨씬 크게 깎인 것들이 있었다.

| 평가 키워드 | 절단 전 | 절단 후 | 손실률 | 걸려 있는 평가 문항 |
|---|---|---|---|---|
| 5영업일 | 18 | 6 | 66.7% | eval-005 (ORBIT), eval-014 (SOLAR) |
| SOLAR | 12 | 6 | 50.0% | eval-014 |
| ORBIT-2 | 12 | 6 | 50.0% | eval-005, eval-011 |
| LYNX-S | 12 | 6 | 50.0% | eval-001, eval-006 |
| AURORA-B | 12 | 6 | 50.0% | eval-002 |

| 평가 키워드 | 절단 전 | 절단 후 | 손실률 | 걸려 있는 평가 문항 |
|---|---|---|---|---|
| VPN / BLUE-9 / LYNX / 개인카드 | — | — | 33.3% | eval-013, eval-009, eval-006, eval-011 |

그리고 평가 문항 기준으로 보면 더 뚜렷하다. 해당 섹션의 학습 샘플 6건 **전부**에서 키워드가 잘려나간 경우가 있었다.

```
eval-001  ORBIT-3, LYNX-S, BLUE-9   → 학습샘플 6/6에서 소실 (5개 중 3개)
eval-013  VPN, 출장, 연차            → 학습샘플 6/6에서 소실 (5개 중 3개)
eval-002  BLUE-7                     → 학습샘플 6/6에서 소실 (5개 중 1개)
eval-005  5영업일                    → 학습샘플 6/6에서 소실 (5개 중 1개)

(키워드 × 샘플) 450쌍 중 48쌍이 절단으로 소실 = 10.7%
```

Known policy는 **Keyword Score가 70% 배점**이고 20문항 중 15문항이 Known이다. 반면 Unknown 5문항의 Safety는 전담 샘플 6건이 온전하고 거절 문장도 439건이 남아 있다. 그래서 처음 가설과 반대로, **타격은 Unknown의 Safety가 아니라 Known의 Keyword Score에 갈 가능성이 높다**고 수정했다.

> 다만 이건 **학습 신호의 손실**이지 **점수 하락의 확정**이 아니다. 모델은 Base Qwen이 원래 갖고 있던 능력으로도 답하고, System Prompt에도 "고유 코드를 정확히 설명하라"는 지시가 들어 있다. 여기까지는 "어디를 의심하며 평가 결과를 봐야 하는가"까지다.

### C-6. 그럼 얼마로 뒀어야 하나

| MAX_LENGTH | 잘리는 샘플 | 비율 | 비고 |
|---|---|---|---|
| 384 (수업 권장, 오늘 사용) | 148건 | 25.4% | 거절 문장이 143건에서 사라짐 |
| 512 (코드 기본값) | 18건 | 3.1% | 메모리·시간 증가 |
| 2,900 (전부 수용) | 0건 | 0% | 최대 2,840 토큰 샘플 1건 때문에 전체가 느려짐 |

512로만 올려도 잘리는 비율이 25.4% → 3.1%로 떨어진다. 코드의 기본값이 애초에 512였다. 수업에서 384를 권장한 건 16GB Mac에서 시간 안에 끝내기 위한 타협이었을 것이고, 실제로 32분에 끝났으니 그 목적은 달성했다. 다만 **"돌아갔다"와 "제대로 학습됐다"는 다른 이야기**라는 걸 오늘 숫자로 봤다.

> 부록 C에서 배운 것 세 가지.
>
> ① **데이터를 안 보고 하이퍼파라미터를 정하면 안 된다.** `SFT_MAX_LENGTH`는 그냥 메모리 설정처럼 보이지만, 실제로는 "학습 데이터의 어느 부분을 버릴 것인가"를 정하는 값이었다. `train_lora.py`의 `inspect_training_sample()`이 이 경고를 해주긴 하는데 첫 샘플 하나만 본다. 첫 샘플이 299 토큰이라 경고가 안 떴다. 한 건이 아니라 분포를 봐야 했다.

② **그럴듯한 가설을 그대로 적으면 안 된다.** "거절 문장이 잘렸으니 Safety가 깎이겠다"는 논리는 매끄러웠지만, 거절 전담 샘플 6건이 안 잘렸다는 걸 세어보니 과장이었다. 세어볼 수 있는 건 세어봐야 한다.

③ **무엇을 기준으로 재는지가 결과를 바꾼다.** 전체 문자열로 재면 "인사팀" 손실이 0%로 나오고, `assistant_only_loss=True`에 맞춰 응답 구간만 재면 20%로 나온다. 같은 데이터인데 손실이 보이거나 안 보인다. 지표를 만들 때 **모델이 실제로 학습하는 범위**와 측정 범위를 맞춰야 한다.

④ **(평가 후 추가) 맞은 것처럼 보이는 결과를 조심해야 한다.** 공식 평가에서 내가 지목한 4문항은 실제로 낮게 나왔다(0.050 vs 0.164). 그대로 "예측 적중"이라고 적을 뻔했다. 그런데 4번 (7)(8)에서 확인했듯, 그 차이를 만든 건 절단이 아니라 `no_repeat_ngram_size`였다. 교란 변수를 지우고 다시 재니 0.200 vs 0.236으로 차이가 사라졌다. **결과가 예측과 같다는 것과, 예측한 이유가 맞다는 것은 다른 이야기다.**

---

## 부록 D. 실무 적용 관점에서 내 생각

### D-1. 오늘 한 일을 한 문장으로 말한다면

사내 인사 규정을 외부 LLM에 보낼 수 없다는 보안 요건 때문에, Qwen2.5-1.5B를 내부에 두고 LoRA(r=16, Attention+MLP 7개 모듈)로 SFT를 적용해 사내 규정 답변 도우미를 만들었습니다. 전체 15.4억 파라미터 중 1.18%인 1,846만 개만 학습해서 32분 만에 70MB 어댑터를 얻었고, Base 대비 20문항 중 18문항에서 우위를 확인했습니다.

다만 절대 점수가 낮아 생성 결과를 직접 열어봤고, 평가 코드의 `no_repeat_ngram_size=3`이 `AURORA-A/B/C/X`처럼 공통 접두어를 반복해야 하는 사내 코드 체계의 출력을 차단하고 있다는 걸 찾았습니다. 이 제약은 정답을 아는 Fine-tuned 모델을 Base보다 더 크게 깎아, SFT 효과를 실제보다 작아 보이게 만들고 있었습니다. Base와 Fine-tuned를 동일 조건으로 재측정하니 Keyword Score 개선폭이 3.1배에서 **4.4배**로, Total Score가 0.113 → 0.419로 정정됐고, 규정에 없는 질문 5건은 전부 만점이 됐습니다.

이렇게 말하려면 각 숫자가 왜 그 값인지 설명할 수 있어야 한다는 게 오늘 실습의 진짜 목적이었던 것 같다. "LoRA를 써봤습니다"는 누구나 말할 수 있지만, "왜 r=16이고, 왜 7개 모듈이고, 그래서 몇 개가 학습됐고, 그게 왜 그 숫자인지"는 돌려보고 뜯어봐야 나온다.

### D-2. 실전이라면 다르게 했을 것들

- **MAX_LENGTH를 데이터 보고 정한다.** 부록 C 그대로다. 실무라면 학습 전에 길이 분포를 찍고 p95 정도를 기준으로 잡겠다. 이 데이터라면 512가 적절했다.

- **평가 코드도 검증 대상에 넣는다.** 오늘 가장 크게 배운 것이다. 점수가 낮으면 모델을 의심하게 되는데, 이번엔 원인이 평가 하네스의 생성 설정이었다. 낮은 점수를 보고하기 전에 **실제 생성 결과를 눈으로 읽어보는 절차**를 반드시 넣겠다. 숫자만 봤으면 "1.5B로는 무리"라는 잘못된 결론을 냈을 것이다.
- **도메인 특성과 생성 설정의 충돌을 확인한다.** `no_repeat_ngram_size`는 반복 억제용으로 흔히 쓰는 설정인데, 공통 접두어를 가진 코드 체계(AURORA-A/B/C/X, BLUE-1/3/5/7/9)를 다루는 도메인에서는 정답을 막는다. 사내 코드 체계가 있는 프로젝트라면 이걸 먼저 점검하겠다.
- **지표를 모델이 실제로 학습하는 범위에 맞춰 잰다.** 부록 C-4가 그 얘기다. `assistant_only_loss=True`인데 전체 문자열로 데이터 품질을 재면 손실이 0%로 보인다. 데이터 검증 스크립트를 짤 때 "모델이 loss를 받는 구간"을 기준으로 재도록 처음부터 맞춰두겠다.
- **평가를 학습보다 먼저 설계한다.** `hr_eval.jsonl`이 Known 15 / Unknown 5로 나뉘어 있고 Unknown에 Safety 50%를 준 게 인상적이었다. "무엇을 잘하면 성공인가"가 데이터로 먼저 정의돼 있었다. 실무에서는 이걸 학습 시작 전에 현업과 합의해야 나중에 "그래서 좋아진 거 맞아요?"에 답할 수 있다.
- **SFT와 RAG를 처음부터 나눠 설계한다.** 규정은 개정된다. 개정 때마다 32분씩 재학습할 수는 없다. 자주 바뀌는 사실(금액, 기한, 한도)은 RAG로 빼고, 잘 안 바뀌는 것(답변 형식, 코드 체계, 거절 규칙)만 SFT로 새기는 게 운영 비용이 낮다.
- **거절을 성능 지표로 관리한다.** 사내 도우미에서 제일 위험한 건 모르는 걸 그럴듯하게 답하는 것이다. 정답률보다 "규정에 없는 질문에 안 지어내는 비율"을 먼저 보겠다. 이 실습 평가표가 그렇게 짜여 있는 게 우연이 아니라고 생각한다.
- **어댑터는 하나로 유지한다.** 인사용·보안용을 따로 만들어 동시에 얹고 싶어지는데, 가이드3 Q14대로 언어모델에서는 간섭이 생긴다. 목적이 여러 개면 데이터셋을 합쳐서 한 번에 학습시키는 쪽을 택하겠다.

#### D-3. 어디에 쓰면 좋을까

오늘 만든 구조가 잘 맞는 조건은 세 가지가 겹치는 곳이라고 정리했다.

1. **데이터가 밖으로 나가면 안 되는 곳** — 인사, 법무, 보안, 의료, 금융 내부 문서
2. **답변의 형식과 태도가 정해져 있는 곳** — 규정 안내, 심사 기준 설명, 표준 응대 스크립트
3. **질문 유형이 반복되는 곳** — 같은 걸 매번 담당자에게 물어보는 업무

반대로 잘 안 맞는 곳도 분명해졌다. 매일 사실이 바뀌는 업무(재고, 시세, 실시간 현황)는 SFT로 새기면 바로 낡는다. 그건 RAG나 DB 조회의 영역이다. 그리고 방대한 새 지식을 넣어야 하는 경우는 SFT가 아니라 CPT다. 오늘 679건으로 한 건 지식 주입이 아니라 **행동 고정**이었다.

#### D-4. 마지막으로 남은 생각

가이드3 맺음말에 "같은 도구를 쓰는 이상, 승부는 트레이싱지를 붙이는 방법이 아니라 그 위에 무엇을 그리느냐에서 갈린다"는 말이 있었다. 오늘 그 말이 실감났던 게 부록 C였다. LoRA 설정(r, alpha,

target_modules)은 코드에 이미 잘 잡혀 있어서 내가 건드릴 게 없었다. 정작 결과를 흔든 건 알고리즘이 아니라 **데이터가 어떻게 잘리는지를 몰랐다는 것**이었다.

LoRA 자체는 이제 표준 도구고, 코드 몇 줄이면 붙는다. 그래서 실무에서 차이를 만드는 건 그 위에서 "우리 도메인의 답변이 어떤 형태여야 하는가"를 정의하고, 그걸 데이터로 만들고, "무엇을 잘하면 성공인가"를 평가로 정의하는 쪽이라고 생각한다. 학습 데이터 679건이 전부 같은 3단 형식으로 통일돼 있던 것도, 평가에서 Unknown에 Safety 50%를 준 것도, 누군가 그 도메인을 알고 설계한 결과였다.

그리고 오늘 가장 크게 남은 건, 내가 세운 예측 두 개가 **둘 다 지지되지 않았다**는 사실 자체다. ①은 명확히 틀렸고, ②는 공식 결과만 봤을 때는 맞아 보였는데 교란 변수를 제거하니 사라졌다. 그런데 그 두 예측을 검증하려고 데이터를 파고든 과정에서, 예측하지 않았던 `no_repeat_ngram_size` 문제를 발견했다. **가설이 틀린 게 손해가 아니라, 틀렸는지 확인하는 과정이 실제 문제를 찾아냈다.**

남은 일은 `SFT_MAX_LENGTH=512`로 다시 학습시켜 BLUE-7이 복구되는지 보는 것이다. 그게 확인되면 절단의 영향을 일화가 아니라 근거로 말할 수 있다. 그리고 Known 문항 Keyword가 동일 조건에서도 0.2267에 그친 건 설정이 아니라 모델의 한계로 보이므로, epoch을 3으로 늘리거나 r을 32로 키웠을 때 이 숫자가 움직이는지도 확인해 보고 싶다.